

Human-AI-Computer Interaction: Measuring Whether AI Strengthens Unaided Judgment

Gianangelo Dichio
Bucket Foundation

2026-10-07

Abstract

A tool that helps a person answer today and leaves them no better at answering alone next week has made them dependent on it. We state a measurement protocol that separates the two outcomes. On one task set, three scores are taken: the person’s **unaided score** H_p , the model’s **AI score** A , and the person’s **joint score** J_p with the model’s answer in view. The **multiplier** is $m_p = J_p / \max(H_p, A)$, the gain is $D = J_p - \max(H_p, A)$, and the unaided test is repeated after seven days to give **retention** $R_p = H_p(t + 7\text{d}) - H_p(t)$ and learning $L = J_p(t + 7\text{d}) - H_p(t + 7\text{d})$. A tool that raises J_p while L stays at or below zero across its interval marks **dependence**. The instrument is the **hai** probe in Bucket’s **bkt** package: a frozen bank of 998 four-choice items across the seven canon branches, of which 68 are held out by an automated distractor review [empirical: packages/bkt/hai/bank.json and review.json, 2026-10-07, bkt hai freeze; bkt hai review]; sessions of 40 items in 20 pairs matched on tier and on whether the model got the pair’s items right; a 20 second think window before the model’s answer appears; no feedback until the retest; guess-corrected scores and a bootstrap over pairs for every interval [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. The learning side of the measurement is the minimum prerequisite learning problem: the catalog is a directed acyclic graph of concepts, a learner’s state is a closed set of mastered concepts, and the least a learner must add to reach a target is the **required set** $R(t, M)$, the target’s **prerequisite closure** minus their mastery, a count proved minimal in Lean 4 under stated assumptions. Three rules will join the two sides, probe items drawn at the learner’s **frontier**, R_p reported against the engine’s predicted recall, and **mastery** updated from the solo half only; all three are design, since the probe code holds no mastery or FSRS logic today. The instrument is built and no probe has been retested, so this report carries no result on a person; the learning system’s own evaluation, 57,060 enumerated planning cases with zero errors and a synthetic prediction experiment, is reported with its source. The report states the planned analysis, paired comparisons stratified by education level, a divergence hypothesis on how reliance on one model narrows the range of answers across a group, and the rule by which the scores decide which tasks Research OS hands to the model and which it keeps with the person.

1 Introduction

Research OS puts a model next to a person on every task in the product: recall of a result, application of a rule, derivation, teaching. Each of those tasks can be left with the person, augmented with the model, or handed to the model. The product has no basis for that choice until it can measure, per person and per task class, what the model adds today and what the person keeps a week later. Field studies report the first number, the lift with the model in hand [1, 2], and a meta-analysis across 106 experiments finds that human and AI together score below the better of the two on average

[3]. The second number, what the person retains, is the one a learning product cannot do without: in a randomized study of high school mathematics, access to a model raised practice scores and lowered unaided exam scores [4]. This report defines both numbers on the same items, describes the instrument that takes them, and says what will be done with them.

2 Related work

Complementary team performance, the case where the pair beats both the person and the model, was named and tested in a user study of AI explanations, which found explanations raised acceptance of the model’s answer whether it was right or wrong [5]. The jagged frontier study randomized consultants to model access and found gains inside the model’s competence and losses outside it [1]; the customer-support study found the largest gains for the least experienced workers [2]. The retention arm of this protocol follows the testing-effect literature, in which an unaided retrieval test is itself the strongest predictor of later recall [6]. The divergence hypothesis of Section 7.4 follows a study in which model assistance raised the rated quality of each writer’s story and lowered the diversity across writers [7]. Intervals use the pair bootstrap [8].

3 Preliminaries

Definition 3.1 (Item and bank). An item is a prompt with four choices, one correct, drawn from the Academy pack and frozen with three distractors fixed under a seed, so the same text is shown to every person and to the model. The bank is versioned by a hash of its items [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. An automated review flags an item whose distractors duplicate each other, contain or overlap the answer by token Jaccard at or above 0.5, answer a second question on the same atom, or whose answer runs to at least twice the length of its longest distractor; a flagged item stays out of probes and scoring until a person clears it [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

Definition 3.2 (Scores). For c correct answers out of n four-choice items, the guess-corrected score is $s = (c/n - 1/4)/(3/4)$, so chance scores 0 and a perfect run scores 1 [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. The **unaided score** H_p is s over a person’s solo items; the **joint score** J_p is s over their pair items; the **AI score** A is s for the model over those same pair items.

Definition 3.3 (Probe). A **probe** is 40 unseen bank items in 20 pairs. Items are grouped by tier and by whether the model got them right, pairs are drawn inside a group, and a seeded coin sends one item of each pair to the solo condition and the other to the pair condition [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. In the pair condition the model’s choice and a rationale of at most 25 words appear after 20 seconds; the person may accept or override, and the acceptance is recorded [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. No correctness is shown on any probe item, and probe items never enter spaced review, until the retest ends [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

Definition 3.4 (Retest). Seven days after a probe, the person answers all 40 of its items again, unaided, in a reshuffled order [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. Scores and answers appear only after the retest.

Definition 3.5 (Multiplier and gain). The **multiplier** is $m_p = J_p / \max(H_p, A)$, reported as null when $\max(H_p, A) < 0.1$; the **gain** is $D = J_p - \max(H_p, A)$ [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. D is the primary statistic and m_p is reported beside it.

Definition 3.6 (Retention and learning). **Retention** is $R_p = H_p(t+7\text{ d}) - H_p(t)$ over the solo items. Learning is $L = J_p(t+7\text{ d}) - H_p(t+7\text{ d})$: at the retest, the score on items first seen with the model minus the score on items first seen alone, both unaided [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

Definition 3.7 (Dependence). **Dependence** is flagged when the 95% interval of $J_p - H_p$ lies above zero and the 95% interval of L includes or lies below zero [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. The gain was there with the model in the room and gone a week later.

4 The learning system

The protocol of Section 3 measures a person on items. The learning system says which items, and what a person's state is before and after. Its contract is the minimum prerequisite learning problem stated in the learning-system brief and checked in Lean 4 [empirical: learning/research-os/learning-system/brief/BRIEF.md, 2026-10-07, human derivation outside Lean], and its per-concept state is the mastery formula of the Academy engine [empirical: src/lib/academy/mastery.ts and fsrs.ts, 2026-10-07, git show dev:src/lib/academy/mastery.ts].

4.1 The minimum prerequisite learning problem

Definition 4.1 (Catalog, closed state, frontier). The catalog is a finite set of concepts V with a directed prerequisite relation $E \subseteq V \times V$, where $p \rightarrow v$ means p is mandatory before v , and the implementation rejects any cycle, so (V, E) is a directed acyclic graph. A knowledge state is a set $K \subseteq V$. K is closed when every immediate prerequisite of every member is a member: $p \rightarrow v$ and $v \in K$ imply $p \in K$. A concept $v \notin K$ is ready, and lies on the **frontier** of K , when every p with $p \rightarrow v$ lies in K [lean: LearningSystem.Closed].

Definition 4.2 (Closure and required set). For a target t , the **prerequisite closure** is the target with every concept on a prerequisite path into it, and the **required set** removes accepted mastery M :

$$C(t) = \{t\} \cup \{p \in V : p \rightsquigarrow t\}, \quad R(t, M) = C(t) \setminus M. \quad (1)$$

$C(t)$ is the inductive predicate **Required** and R is its conjunction with $\neg \text{known}$ [lean: LearningSystem.Required].

Definition 4.3 (Transition and feasible sequence). A step adds one fresh ready concept and keeps everything already known:

$$K_{i+1} = K_i \cup \{v_i\}, \quad v_i \notin K_i, \quad (p \rightarrow v_i \Rightarrow p \in K_i). \quad (2)$$

A finite list $\pi = (v_1, \dots, v_n)$ is feasible from K_0 when every step satisfies Equation (2); it reaches t when $t \in K_0$ or t occurs in π [lean: LearningSystem.Path, LearningSystem.After].

Definition 4.4 (Cost model). Each concept carries a fixed cost $w(v) \in \mathbb{N}$, zero allowed. The effort of a sequence and the minimum effort to a target are

$$W(\pi) = \sum_{v \in \pi} w(v), \quad c^*(M, t) = \sum_{v \in R(t, M)} w(v), \quad (3)$$

and the unit-cost distance is $d^*(M, t) = \min\{|\pi| : \pi \text{ feasible from } M_C, \pi \text{ reaches } t\}$, with $M_C = M \cap C(t)$ the mastery restricted to the closure [lean: LearningSystem.Effort].

A, the AI alone, is scored once per frozen item before any probe

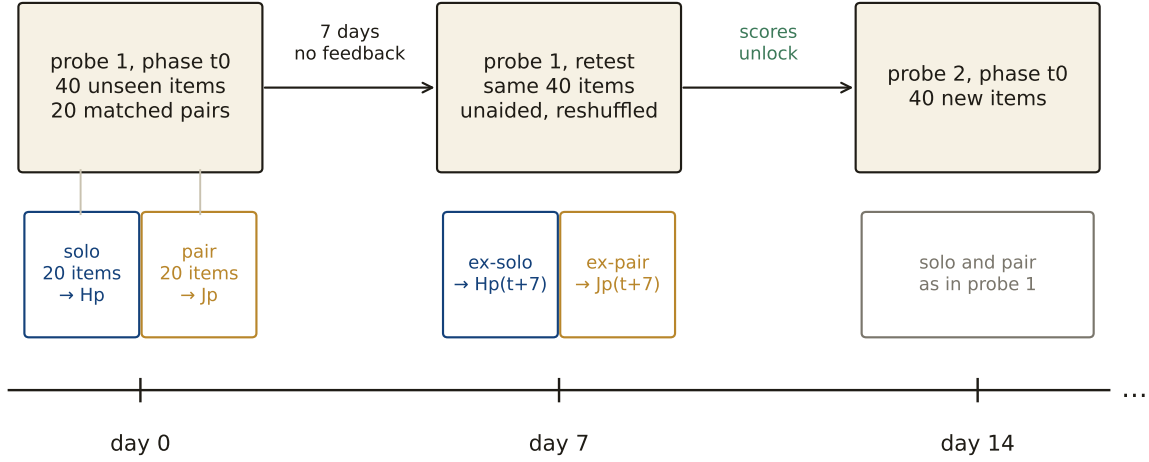


Figure 1: The session timeline. Each probe splits 40 unseen items into solo and pair conditions on day 0, is retested unaided in full on day 7, and unlocks its scores only then; the next probe draws 40 new items. *A* is scored on every frozen item once, before any person sees it [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

Reading the figure. Every number in this box: [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

A probe is 40 items in 20 pairs; one item of each pair goes to the solo condition and gives H_p , the other to the pair condition and gives J_p . The retest on day 7 reuses the same 40 items, unaided and reshuffled, and gives $H_p(t+7d)$ on the ex-solo half and $J_p(t+7d)$ on the ex-pair half. Nothing is shown to the person between day 0 and the end of the retest: no correctness, no score, and no probe item enters spaced review. The day 14 probe draws 40 items the person has never seen, so two probes test different items and a change in H_p across probes mixes the person with the items, which is why every within-probe statistic is a matched comparison and why no trend is read before 5 retested probes. The 20 second window before the model’s answer appears is the same on every pair item, and the model’s answers were fixed before the first probe, so *A* is a property of the bank version.

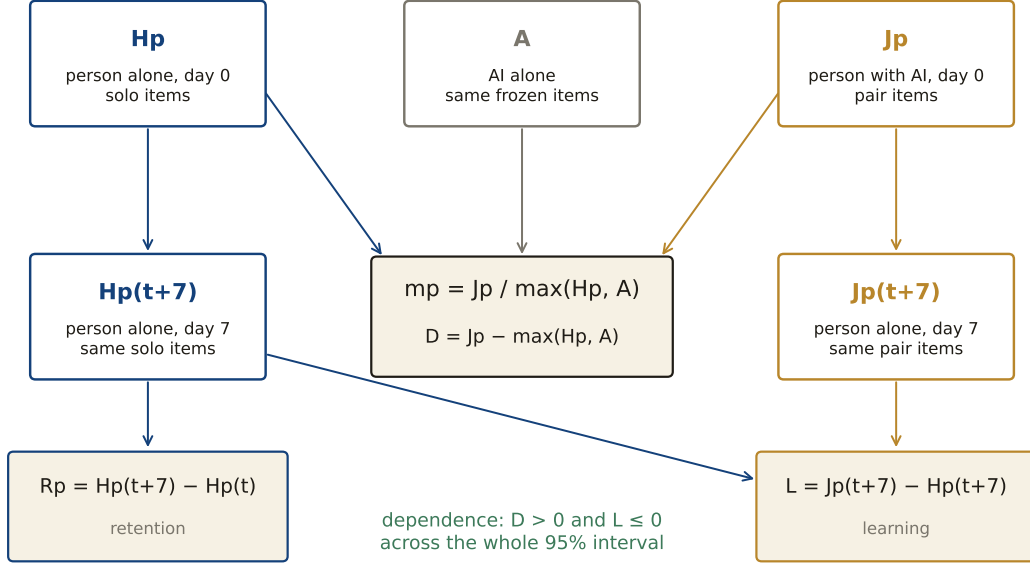


Figure 2: The four scores and the statistics derived from them. H_p , A and J_p combine into the multiplier and the gain; the day 7 scores give retention on the solo items and learning on the pair items; dependence is the pattern where the gain is positive and learning is not [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

Reading the figure. Every number in this box: [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

Three scores are taken on day 0 and two on day 7, all on the guess-corrected scale $s = (c/n - 1/4)/(3/4)$, so each runs from $-1/3$ at zero correct through 0 at chance to 1 at a perfect run. $m_p = J_p / \max(H_p, A)$ is reported as null when $\max(H_p, A) < 0.1$, since a ratio over a near-zero denominator carries no information, and $D = J_p - \max(H_p, A)$ is the primary statistic. R_p is a within-item difference on the 20 solo items and L is a between-half difference at the day 7 retest, so the practice effect of the retest falls on both halves of L and cancels to first order. Dependence needs the whole 95% interval of $J_p - H_p$ above zero and the whole interval of L at or below zero; the two intervals are percentile bootstraps over the 20 matched pairs with 2,000 draws, and an interval is withheld when fewer than half the draws give a finite statistic.

The brief states five assumptions the results rest on [empirical: learning/research-os/learning-system/brief/BRIEF.md, 2026-10-07, human derivation outside Lean]: every encoded edge is mandatory and the snapshot holds every prerequisite the target needs (A1); $C(t)$ is finite and the planner constructs a prerequisite-first order (A2); for each mastered $v \in C(t)$ every immediate prerequisite of v is mastered, so M_C is closed (A3); a step adds one concept and nothing is forgotten during the episode (A4); costs are fixed natural numbers shared across the sequences compared (A5).

Lemma 4.5 (Closure is preserved). *If K is closed and v is ready in K , then $K \cup \{v\}$ is closed; by induction every state reached by a feasible sequence from a closed state is closed [lean: LearningSystem.after_closed].*

Lemma 4.6 (Target mastery entails ancestor mastery). *If a closed state K contains t , then $C(t) \subseteq K$ [lean: LearningSystem.required_known].*

Theorem 4.7 (Necessary additions). *From a closed M_C , every feasible sequence π that reaches t contains every member of $R(t, M)$, so $|\pi| \geq |R(t, M)|$ [lean: LearningSystem.remaining_necessary].*

Proof. By Lemma 4.5 the final state is closed; by Lemma 4.6 it contains $C(t)$. The final state is M_C together with the additions of π , and a member of R is absent from M_C , so it is one of the additions. $\square$

Theorem 4.8 (Attained minimum). *Let π list each element of $R(t, M)$ once in an order where every prerequisite of v_i lies in M_C or earlier in π . Then π is feasible, reaches t , and $d^*(M, t) = |R(t, M)|$ [lean: LearningSystem.minimum_unit_distance]. With fixed costs the same π attains $c^*(M, t)$ [lean: LearningSystem.minimum_weighted_effort].*

Corollary 4.9 (Mastery shrinks the required set). *If $M_1 \subseteq M_2$ then $R(t, M_2) \subseteq R(t, M_1)$ and $|R(t, M_2)| \leq |R(t, M_1)|$ [lean: LearningSystem.distance_antitone].*

Figure 3 shows the diamond that separates the theorem from chain planning: $F \rightarrow A, F \rightarrow B, A \rightarrow T, B \rightarrow T$ with empty mastery. The chain F, A, T has three nodes and never reaches T , since after $\{F, A\}$ the target is still blocked by B ; the feasible sequence F, A, B, T has four transitions, equal to $|R(T, \emptyset)|$, and F, B, A, T is a second optimum [lean: LearningSystem.diamond_minimum, diamond_chain_insufficient]. The planner's running time with adjacency lists is $O(|C(t)| + |E_C|)$ for E_C the edges inside the closure [empirical: learning/research-os/learning-system/brief/BRIEF.md, 2026-10-07, human derivation outside Lean]; this bound is a human argument outside Lean.

4.2 The Lean contract

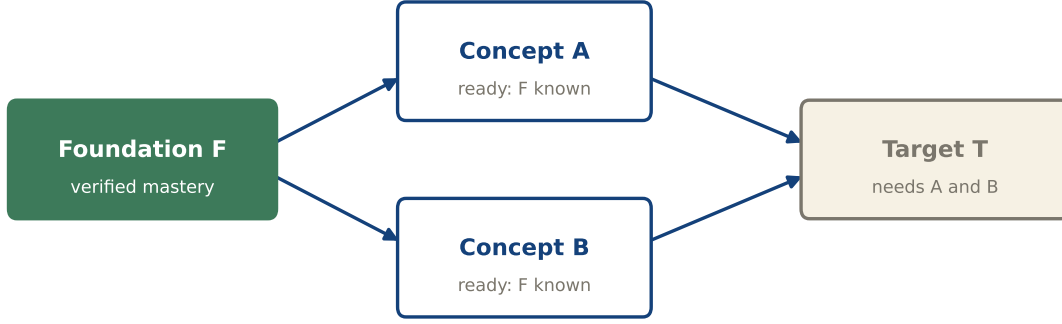
The learning-system source is `lean/Bucket/Learning.lean`, a copy of `learning/research-os/learning-system/lean/LearningSystem.lean` built by `make lean` under Lean 4.33.1 with bundled `Std` and no other dependency [lean: papers/human-ai-multiplier/lean]. The four statements the section rests on are quoted here; Appendix B lists the rest.

```
def Closed (edge : a -> a -> Prop) (known : a -> Prop) : Prop :=
  forall p v, edge p v -> known v -> known p

inductive Required (edge : a -> a -> Prop) (target : a) : a -> Prop
  where
  | target : Required edge target target
  | prerequisite {p v} : edge p v -> Required edge target v -> Required
    edge target p
```

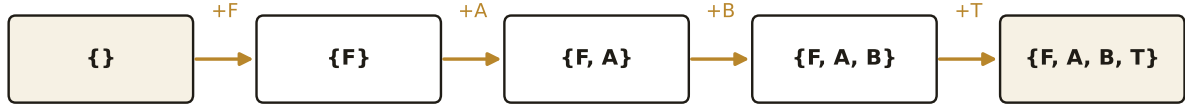
PREREQUISITE CHART FOR TARGET T

4 required / 1 verified / 3 remaining



filled: known outline: ready, the frontier where probe items are drawn tinted: blocked

SHORTEST KNOWLEDGE-STATE PATH FROM EMPTY MASTERY



4 transitions = $|R(T, \{\})| = 4$ required concepts

the chain F, A, T has 3 nodes and never reaches T : after $\{F, A\}$, T is still blocked by B

swapping A and B gives a second optimum; the minimum is a count, the order is a certificate

Figure 3: The prerequisite chart for a target T with one verified foundation, two ready concepts and a blocked target, and the shortest knowledge-state path from empty mastery through the same diamond [empirical: `learning/research-os/learning-system/brief/BRIEF.md`, 2026-10-07, human derivation outside Lean].

Reading the figure. Every number in this box: [empirical: `learning/research-os/learning-system/brief/BRIEF.md`, 2026-10-07, human derivation outside Lean], with the diamond facts checked in `lean/Bucket/Learning.lean`.

The chart has four required concepts, one verified and three remaining, which is $|C(T)| = 4$, $|M_C| = 1$ and $|R(T, M)| = 3$ for this learner. The two outlined concepts are the frontier: F is known and is the only prerequisite of each, so both are ready, and these are the concepts a probe will draw items from. T is tinted because A and B are both mandatory and neither is known. The lower panel starts from empty mastery, so $|R(T, \emptyset)| = 4$ and the path has four transitions: each arrow adds one ready concept, and the state after each arrow is closed. The chain F, A, T counts three nodes and fails at its third step because $B \notin \{F, A\}$; counting nodes on a path through the graph is the error the theorem rules out, since the quantity that is minimal is the count of the required set and the order is a certificate that the count is attained. Swapping A and B gives a second optimum, so the minimum is unique and the order is not. Fixed costs 2, 3, 5, 4 on F, A, B, T give $c^* = 14$ from empty mastery and 12 once F is known.

```

theorem remaining_necessary {edge : a -> a -> Prop} {known : a -> Prop}
  {target v : a} {steps : List a} (hc : Closed edge known)
  (hp : Path edge known steps) (ht : After known steps target)
  (hr : Required edge target v) (hu : ~ known v) : v in steps

theorem minimum_unit_distance {edge : a -> a -> Prop} {known : a -> Prop}
}
  {target : a} {order : List a} (hc : Closed edge known)
  (en : Enumerates edge known target order) (earlier : Earlier edge
    known order) :
  Path edge known order /\ After known order target /\
    forall steps, Path edge known steps -> After known steps target ->
      order.length <= steps.length

theorem minimum_weighted_effort {edge : a -> a -> Prop} {known : a ->
  Prop}
  {target : a} {order : List a} (weight : a -> Nat) (hc : Closed edge
    known)
  (en : Enumerates edge known target order) (earlier : Earlier edge
    known order) :
  Path edge known order /\ After known order target /\
    forall steps, Path edge known steps -> After known steps target ->
      Effort weight order <= Effort weight steps

```

What is proved. Lemma 4.5, Lemma 4.6, Theorem 4.7, Theorem 4.8 and Corollary 4.9 are the Lean theorems after_closed, required_known, remaining_necessary, minimum_unit_distance, minimum_weighted_effort and distance_antitone; restricted_closed and restricted_remaining justify the restriction to M_C ; the diamond is five theorems on a four-element inductive type; coverage_monotone and squared_extent_monotone prove the natural-number cases of the coverage results below. The axiom audit of all ten named theorems reports dependencies empty or within propext, Classical.choice and Quot.sound, with no sorry, custom axiom or native_decide [empirical: learning/research-os/learning-system/lean/check-output.txt, 2026-10-07, bash lean/check.sh].

What is deferred. The enumeration and the ready-order certificate are inputs to the theorem; Lean does not construct them from a graph, verify a topological sort, or prove the TypeScript planner. The real-valued coverage propositions of Section 4.3 are human proofs. Nothing in Lean touches the mastery formula, FSRS, assessment validity or any learner outcome; those live in tests and in the protocol of Section 5.

4.3 The knowledge region

Definition 4.10 (Coverage). Fix a catalog V of factored concepts and a set J of primitive axes; each concept v has a nonempty set of distinct supports $S(v) \subseteq J$. Its allocation and each axis denominator are

$$A_{vj} = \frac{\mathbf{1}[j \in S(v)]}{|S(v)|}, \quad d_j = \sum_{v \in V} A_{vj} > 0. \quad (4)$$

With $m(v) \in \{0, 1\}$ the confirmed flag, the confirmed coordinate on axis j , the catalog `coverage` and the radial extent are

$$g_j = \frac{\sum_v A_{vj} m(v)}{d_j}, \quad G = \frac{\sum_j d_j g_j}{\sum_j d_j}, \quad r = \sqrt{\frac{\sum_j d_j g_j^2}{\sum_j d_j}}. \quad (5)$$

Proposition 4.11 (Bounds and identity). *Every $g_j \in [0, 1]$, $G = \sum_v m(v)/|V|$, and $G \leq r \leq \sqrt{G}$ [empirical: `learning/research-os/learning-system/brief/BRIEF.md`, 2026-10-07, human derivation outside Lean].*

Proof. Each row of A sums to one, so interchanging sums gives $\sum_j d_j = |V|$ and $\sum_j d_j g_j = \sum_v m(v)$. The weighted variance $\sum_j d_j (g_j - G)^2 / \sum_j d_j = r^2 - G^2 \geq 0$ gives $G \leq r$; $g_j^2 \leq g_j$ on $[0, 1]$ gives $r^2 \leq G$. $\square$

Proposition 4.12 (Monotonicity). *With V , S and d fixed, if no flag $m(v)$ decreases then no g_j , G or r decreases [empirical: `learning/research-os/learning-system/brief/BRIEF.md`, 2026-10-07, human derivation outside Lean]. The natural-weight coverage sum and the integer squared extent are the cases proved in Lean [lean: `LearningSystem.coverage_monotone, squared_extent_monotone`].*

Figure 4 draws the worked fixture: $S(a) = \{a\}$, $S(b) = \{b\}$, $S(t) = \{a, b\}$, so $d_a = d_b = 3/2$; mastering a gives $g = (2/3, 0)$, $G = 1/3$ and $r^2 = 2/9$; adding b gives $(2/3, 2/3)$, $G = 2/3$ and $r^2 = 4/9$; adding t gives $G = r = 1$ [empirical: `learning/research-os/learning-system/analysis/results/metrics.json`, seed 20260925, python3 `analysis/evaluate.py`]. r and G measure different things, since profiles with the same G can differ in r , and neither carries a meaning of intelligence or general competence.

4.4 Mastery and retention as implemented

Definition 4.13 (FSRS-5 retrievability). After a review that left a concept at stability $S > 0$ days, the predicted recall t days later is

$$R(t, S) = \left(1 + F \frac{t}{S}\right)^{-1/2}, \quad F = 0.9^{-2} - 1 = \frac{19}{81}, \quad (6)$$

so $R(S, S) = 0.9$ by construction and the next review is scheduled at the interval where R falls to the requested retention 0.9 [empirical: `src/lib/academy/mastery.ts` and `fsrs.ts`, 2026-10-07, git show dev:src/lib/academy/mastery.ts]. Stability and difficulty update after each rating by the FSRS-5 recurrences with the 19 default weights $w_0 = 0.40255$ through $w_{18} = 0.6621$ [empirical: `src/lib/academy/mastery.ts` and `fsrs.ts`, 2026-10-07, git show dev:src/lib/academy/mastery.ts].

Definition 4.14 (Mastery). Proficiency is a logistic function of the concept's ability estimate θ against the apply-depth offset $b_{\text{apply}} = -0.2$, retention is retrievability at the 90-day horizon, and `mastery` is their product:

$$P = \sigma(\theta - b_{\text{apply}}), \quad R = R(90, S), \quad M = P^\alpha R^\beta, \quad \alpha = \beta = 1, \quad (7)$$

clipped to $[0, 1]$, with $M = 0$ when either factor is zero and $M = 1 - e^{-S/21}$ when no proficiency record exists; a concept counts as mastered at $M \geq 0.7$ [empirical: `src/lib/academy/mastery.ts` and `fsrs.ts`, 2026-10-07, git show dev:src/lib/academy/mastery.ts].

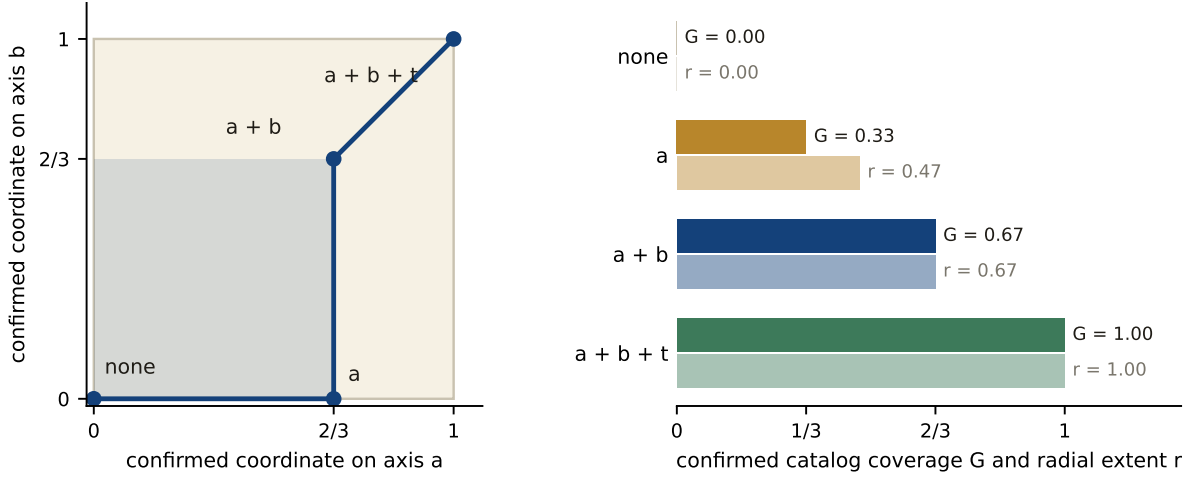


Figure 4: The knowledge region on the three-concept fixture: the path of confirmed coordinates (g_a, g_b) as a , then b , then t are confirmed, and the catalog coverage G with the radial extent r at each milestone [empirical: `learning/research-os/learning-system/analysis/results/metrics.json`, seed 20260925, `python3 analysis/evaluate.py`].

Reading the figure. Every number in this box: [empirical: `learning/research-os/learning-system/analysis/results/metrics.json`, seed 20260925, `python3 analysis/evaluate.py`].

Both denominators are $3/2$, since a and b each receive allocation 1 from their own concept and $1/2$ from t . Confirming a alone moves the point to $(2/3, 0)$: the axis a gains 1 of its $3/2$, and axis b nothing. $G = 1/3$ is one confirmed concept of three, which is the identity of Proposition 4.11, and $r = \sqrt{2/9} = 0.47$ sits between $G = 0.33$ and $\sqrt{G} = 0.58$. Confirming b gives $(2/3, 2/3)$, $G = 2/3$ and $r = 2/3$, the point where the two agree because the coordinates are equal and the variance term vanishes. Confirming t fills both axes, since t contributes its remaining $1/2$ to each, and $G = r = 1$. The shaded square marks the region reachable before t : no flag on a or b alone can push either coordinate past $2/3$. The saved run also records duplicate invariance, repeating a inside the support of t changes nothing after set deduplication, and monotonicity across all 2^3 flag vectors, both **true** in the metrics file. The coordinates describe where confirmed concepts fall in a declared basis for one catalog version; a larger catalog or a new basis changes the denominators and ends the comparison.

The exponents are the `MASTERY_ALPHA` and `MASTERY_BETA` constants of `src/lib/academy/mastery.ts`, both 1.0, so the formula as shipped is the plain product and the exponents are a hook for a later fit rather than a fitted parameter [empirical: `src/lib/academy/mastery.ts` and `fsrs.ts`, 2026-10-07, `git show dev:src/lib/academy/mastery.ts`]. Figure 5 shows the two factors. The set M in Section 4.1 is the accepted mastery that the planner reads; the mastery formula here is the engine’s per-concept signal, and the brief’s A3 is the rule that a concept enters M only when its evidence and its prerequisites’ evidence satisfy the contract. The mapping from $M \geq 0.7$ to membership in M is a product decision that no theorem covers.

5 Connecting the learning system to the protocol

What exists in `packages/bkt/src/hai` is the frozen item bank, the solo and pair conditions, the seven-day retest and the scoring of Section 3; the probe holds no prerequisite graph, no FSRS state and no mastery code, and its draw groups items by tier and by the model’s correctness only [empirical: `packages/bkt/src/hai`, 2026-10-07, `git show dev:packages/bkt/src/hai`]. The three rules below are design. Each names what the protocol will do once the Academy’s prerequisite graph and engine state are bound to the bank’s atoms by version. Figure 6 draws them over a learner’s knowledge region.

Items at the frontier. The protocol will draw the probe’s 40 items from atoms at the learner’s [frontier](#): concepts outside their accepted mastery M whose immediate prerequisites are all in M , the outlined nodes of Figure 3. An item on a known atom measures recall of something already mastered and an item on a blocked atom measures a prerequisite gap; an item at the frontier is one the person could answer from what they know, which is where a model’s help can either teach or substitute. Within the frontier set, pairs will still be matched on tier and on the model’s correctness as in Section 6, A stays the per-item score fixed before any draw, J_p will use the pair half of the same draw, and the retest will reuse all 40, so the frontier rule changes the population of items and none of the definitions.

Retention against the predicted recall. R_p is an observed change in recall over seven days. For each solo item the engine will carry a prediction, $R(7, S)$ from Equation (6) with S the atom’s stability at the probe, and the report will print the predicted mean recall beside the observed one. A retest under the prediction on the pair half and at it on the solo half would be the dependence pattern of Definition 3.7 seen from the engine’s side; a retest above the prediction on both halves would be the testing effect [6]. Probe items stay out of spaced review until the retest ends, so S would be frozen across the seven days and the prediction fixed before the observation.

What the decision rule will read. The cells of Table 2 are read per task class, and the frontier rule makes a task class a set of atoms at a depth on frontier concepts. A class in the augment cell will have its atoms’ M updated from the pair half as well as the solo half; a class in the dependence cell will update M from the solo half only, so a model’s answer could never raise a concept into accepted mastery on its own.

6 Method

The instrument. The probe lives in `packages/bkt/src/hai/` and runs inside the `bkt` terminal app, offline, with answers sealed in the same encrypted store as the person’s other attempts; nothing

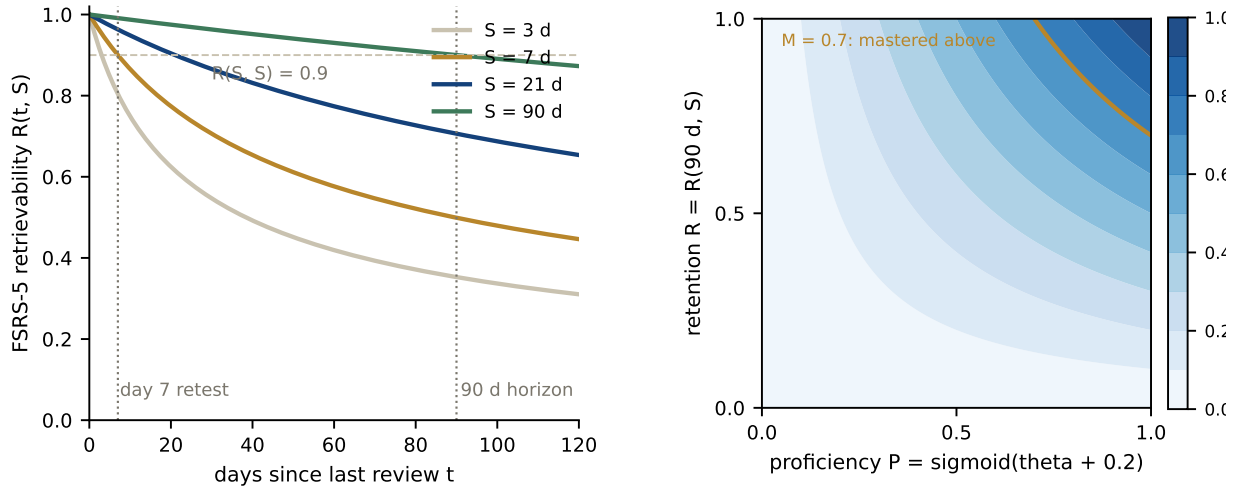


Figure 5: FRS-5 retrievability over 120 days for four stabilities, with the day 7 retest and the 90-day mastery horizon marked, and the mastery surface $M = PR$ over proficiency and retention with the 0.7 mastered contour [empirical: `src/lib/academy/mastery.ts` and `fsrs.ts`, 2026-10-07, `git show dev:src/lib/academy/mastery.ts`].

Reading the figure. Every number in this box: [empirical: `src/lib/academy/mastery.ts` and `fsrs.ts`, 2026-10-07, `git show dev:src/lib/academy/mastery.ts`], evaluated from Equation (6) and Equation (7).

Every curve crosses 0.9 at $t = S$, since F is chosen so that $R(S, S) = 0.9$. At the day 7 retest a concept at stability 3 days predicts recall 0.80, at 7 days 0.90, at 21 days 0.96 and at 90 days 0.99; at the 90-day horizon the same four predict 0.35, 0.50, 0.71 and 0.90, and those are the R values that enter mastery. The surface on the right is the product, so $M = 0.7$ is the hyperbola $PR = 0.7$: a concept with $P = 1$ is mastered at $R \geq 0.7$, which needs $S \geq 20.3$ days at the 90-day horizon, and a concept with $R = 1$ needs $P \geq 0.7$, which is $\theta \geq 0.65$ against the apply offset. With $\alpha = \beta = 1$ the two factors trade one for one in log space; a different pair of exponents would tilt the contour and nothing in the shipped code sets them to anything else. The retention axis is a prediction from the FRS-5 state, and the protocol's R_p will be the observation that tests it.

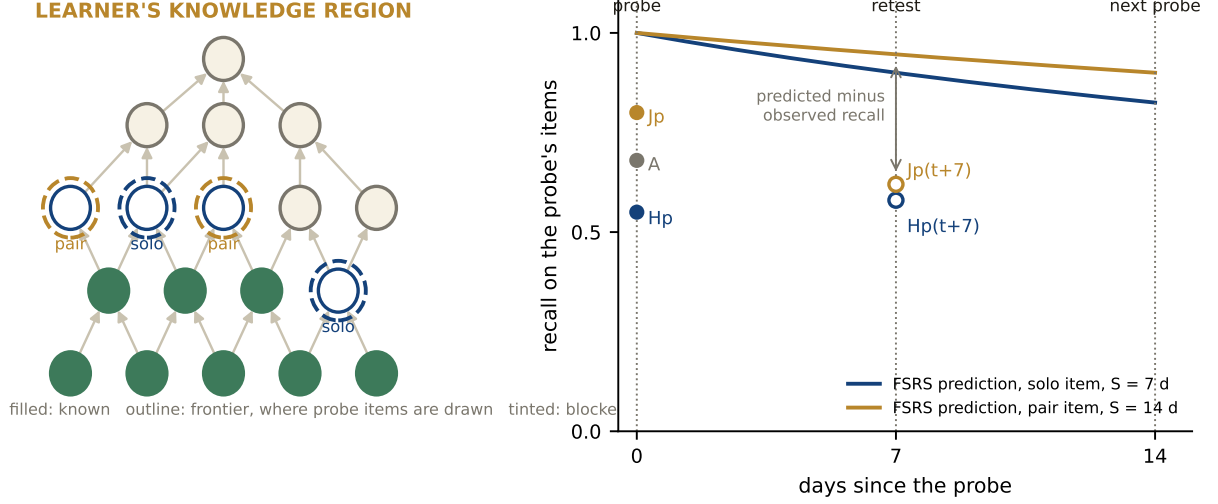


Figure 6: The protocol over a learner's knowledge region. Left: a layered catalog with known, frontier and blocked concepts, and four probe items drawn at the frontier, two solo and two pair. Right: the timeline of one probe with the day 0 scores, the day 7 retest and the FSRS-5 predicted recall for a solo and a pair item. The points are placed to show the pattern and carry no measured value [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

Reading the figure. Every number in this box: a schematic; the two curves are Equation (6) at $S = 7$ and $S = 14$ days, and the point heights are illustrative.

The left panel has 18 concepts in five layers: 8 known, 4 at the frontier, where every incoming arrow comes from a known node, and 6 blocked. The four dashed rings are the probe draw, and every one sits on a frontier node; a real probe will draw 40 items across the frontier atoms of all seven branches. The right panel places H_p , A and J_p on day 0 and the two retest scores on day 7 at heights chosen to show one dependence pattern: J_p above both single agents on day 0, and on day 7 the ex-pair half back near the ex-solo half, so $D > 0$ and $L \approx 0$. The curves are what the engine predicts for recall of an item reviewed at the probe at stability 7 or 14 days: 0.90 and 0.95 on day 7. The gap marked between the pair curve and the ex-pair retest is the quantity the protocol will report beside R_p , predicted minus observed recall. The timeline is fixed by the probe, with the next draw at day 14, and the prediction is fixed on day 0 because probe items are held out of spaced review until the retest ends.

leaves the device, and the person can export or wipe their data [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. **bkt hai freeze** writes the bank, **bkt hai review** writes the flags, **bkt hai score** sends the bank to the model through a batch job and stores each answer with its rationale, model id and date, **bkt hai** runs a probe or its retest, and **bkt hai report** prints the scores as sentences [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

The AI condition. A is one run of **claude-opus-5-5** at low effort over each frozen item, the reply constrained to a JSON object holding a letter and a rationale, with 400 output tokens at most [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. A reply that fails to parse, names no valid letter or hits the token cap is counted as malformed and left out of A and of pair matching [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. The model sees the same four choices the person sees, so A is a property of the item and the bank version.

Matching. Pairs are matched on tier and on the model’s correctness so that H_p and J_p are taken on items of the same difficulty and the same model behaviour. A matched pair in which the model is wrong measures whether the person overrides it; a pair in which it is right measures whether they accept it.

Intervals. Every interval is a 95% percentile bootstrap over matched pairs, 2,000 draws under a fixed seed, resampling pairs and keeping both items of each [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. An interval is withheld when fewer than half the draws yield a finite statistic [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. For a half-width of 0.10 on $J_p - H_p$ at a true score near 0.5, the guess-corrected scale needs about 342 items per condition; a half-width of 0.15 needs about 152 [empirical: packages/bkt/src/hai/stats.ts halfWidthItems, 2026-10-07, evaluated by hand]. The 998-item bank caps one person at 499 items per condition [empirical: packages/bkt/hai/bank.json and review.json, 2026-10-07, bkt hai freeze; bkt hai review], so the design target is a half-width of 0.10 after 17 probes, and no trend is shown before 5 retested probes [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai].

7 Design

7.1 Status

The bank is frozen at 998 items: 482 recall, 240 apply, 177 derive, 90 teach and 9 conceptual, across biophysics (281), information (120), mind (120), cosmology (118), mathematics (110), physics (110), chemistry (110) and learning-to-learn (29) [empirical: packages/bkt/hai/bank.json and review.json, 2026-10-07, bkt hai freeze; bkt hai review]. The review flags 68 items and none has been cleared [empirical: packages/bkt/hai/bank.json and review.json, 2026-10-07, bkt hai freeze; bkt hai review]. No AI scores have been collected, no probe has been run, and **bkt hai report** prints that no probe has been retested [empirical: packages/bkt/hai/bank.json and review.json, 2026-10-07, bkt hai freeze; bkt hai review]. Every number above is a property of the instrument or a planning quantity. There are no values of H_p , A , J_p , m_p , R_p or L to report.

7.2 Learning system results

The learning system has two saved evaluations, both in **learning/research-os/learning-system/analysis/results/metrics.json**, written by **analysis/evaluate.py** under seed 20260925 with

Python 3.13.13 and NumPy 2.5.1 [empirical: learning/research-os/learning-system/analysis/results/metrics.json, seed 20260925, python3 analysis/evaluate.py]. Neither involves a person.

Graph experiment. Every edge subset under a fixed topological labelling for one through five nodes gives 1,099 DAGs; enumerating downward-closed mastery sets and targets over them gives 57,060 cases [empirical: learning/research-os/learning-system/analysis/results/metrics.json, seed 20260925, python3 analysis/evaluate.py]. The candidate planner computes the unmet closure and a prerequisite-first order; the reference is a breadth-first search over ready additions that shares no code with the candidate. Across all 57,060 cases the distance error $\Delta = |\pi_{\text{cand}}| - d_{\text{BFS}}$, the omitted-required count $|R \setminus \text{set}(\pi)|$ and the readiness-error count were all zero [empirical: learning/research-os/learning-system/analysis/results/metrics.json, seed 20260925, python3 analysis/evaluate.py]. The fixtures return the diamond sequence of length 4, a shortcut of length 3, a mastered target at 0, an unmastered root at 1, and typed failures for a cycle, a mastery conflict, an unreviewed foundation and a missing node; the chain counterexample records 3 chain nodes against 4 minimum learning nodes with concept 2 omitted and a readiness failure at the target [empirical: learning/research-os/learning-system/analysis/results/metrics.json, seed 20260925, python3 analysis/evaluate.py].

Statistical experiment. 1,200 synthetic learners with eight items each are split by learner into 400 training, 200 calibration and 600 test, so the test set is $N = 4,800$ responses [empirical: learning/research-os/learning-system/analysis/results/metrics.json, seed 20260925, python3 analysis/evaluate.py]. Three logistic models are fitted with ridge 0.01 and recalibrated on the calibration learners before scoring: intercept only, item effects, and item effects with synthetic axis and prerequisite covariates. Table 1 gives the saved Brier scores and the two residual correlations; Figure 7 draws them. The planted pair’s 97.5% bootstrap interval is $[0.24379, 0.41299]$ and the control’s is $[-0.16685, 0.00182]$, from 2,000 learner bootstraps and 2,000 learner permutations with Holm correction over the two tests [empirical: learning/research-os/learning-system/analysis/results/metrics.json, seed 20260925, python3 analysis/evaluate.py].

Table 1: Saved results of the learning-system statistical experiment on synthetic data [empirical: learning/research-os/learning-system/analysis/results/metrics.json, seed 20260925, python3 analysis/evaluate.py].

Model or diagnostic	Saved value
Intercept-only prediction	Brier 0.24633
Item-intercept baseline	Brier 0.23475
Axis and prerequisite covariates	Brier 0.18649
Reduction over the item baseline	0.04826
Planted residual pair	correlation 0.33463, Holm $p = 0.00100$
Control residual pair	correlation -0.07997 , Holm $p = 0.05497$

The generator that produced the learners also contains the axis and prerequisite predictors, so their Brier advantage is expected by construction and shows that the pipeline recovers what it planted [empirical: learning/research-os/learning-system/brief/BRIEF.md, 2026-10-07, human derivation outside Lean]. The control’s non-rejection is no evidence of independence. No number in this subsection is a measurement of a person, a real prerequisite relation or a calibrated mastery estimate.

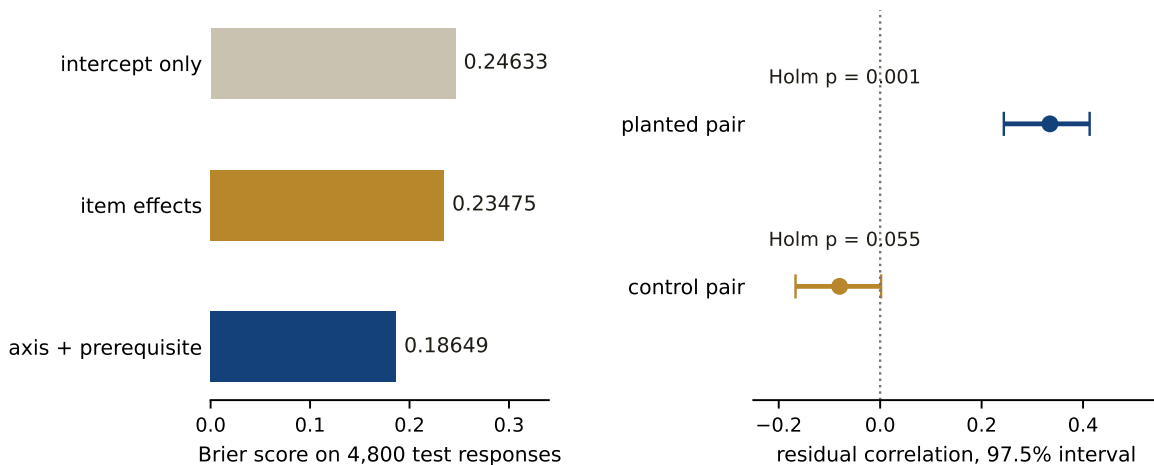


Figure 7: The synthetic statistical experiment: Brier scores of the three models on the 4,800 test responses, and the two prespecified residual correlations with their Bonferroni 97.5% bootstrap intervals and Holm-adjusted permutation p -values [empirical: `learning/research-os/learning-system/analysis/results/metrics.json`, seed 20260925, python3 `analysis/evaluate.py`].

Reading the figure. Every number in this box: [empirical: `learning/research-os/learning-system/analysis/results/metrics.json`, seed 20260925, python3 `analysis/evaluate.py`].

The three bars are Brier scores, mean squared error of a predicted probability against a binary outcome, so lower is better and 0.25 is the score of predicting 0.5 everywhere: the intercept model at 0.24633 sits just under it because the base rate is 0.563 rather than 0.5. Item effects take it to 0.23475 and the axis and prerequisite covariates to 0.18649, a reduction of 0.04826 that the generator built in. The right panel shows the residual correlation of two item pairs after the full model: the planted pair, whose shared latent factor the generator added, at 0.33463 with an interval of [0.24379, 0.41299] clear of zero and Holm $p = 0.00100$; the control pair at -0.07997 with an interval of $[-0.16685, 0.00182]$ that touches zero and Holm $p = 0.05497$. The intervals condition on the fitted model and omit fitting uncertainty, the 4,800 responses are clustered eight per learner, and the item bank is shared across the splits, so held-out learners supply no evidence on unseen item families.

7.3 Planned analysis

Per person, the report is D with its interval, m_p beside it, R_p , L and the dependence flag, each with the number of pairs behind it, and the number of malformed model replies left out. Within a person, D and L are also broken out by tier and by whether the model was right on the pair, so a positive D driven by accepting correct answers is told apart from one driven by overriding wrong ones.

Across people, every comparison is paired: each person contributes their own H_p , J_p and retest scores on their own matched items, and the group statistic is the mean of the per-person differences with a bootstrap over people. Participants are stratified by highest education level reported at consent, in four bands: secondary, bachelor’s, master’s, doctoral. The question for the strata is whether D and L move together or apart with prior training, since the field studies cited above found the largest lift for the least experienced [2] and the learning study found that unaided exam scores fell where the model gave answers and held where it gave hints [4]. The education band is stored at consent and nowhere else, and a band with fewer than 20 people reports counts only.

7.4 The divergence hypothesis

The hypothesis is that a group of people answering with the same model converges on the model’s answers and the model’s phrasings, so the range of ideas the group produces narrows even where each person’s score rises. Four-choice items cannot show this, since the answer space is fixed, so the test needs open items: a second bank of short free-response prompts in the derive and teach tiers, each with a rubric. Each person answers a matched half alone and a half with the model’s draft in view. The responses are embedded with the same sentence encoder Bucket uses for the solvability atlas, and divergence is measured as the mean pairwise cosine distance among the group’s answers to one prompt, solo against pair, with a bootstrap over people. A second measure is the number of distinct answer clusters at a fixed distance threshold. The prediction, following Doshi and Hauser [7], is a higher rubric score and a lower pairwise distance in the pair condition. A product that raises every person’s score while collapsing the group’s spread has optimized the wrong thing for a research community, whose value is in the answers nobody else gave.

7.5 Decision rule for Research OS

The scores decide, per task class, what Research OS does with the model. Table 2 gives the rule. A class where D is above zero and L is at or above zero is augmented: the model’s answer stays in view and the person’s own attempt stays first. A class where D is above zero and L is below zero, the dependence pattern, is augmented only when the task is a means and never when it is a learning target; in the Learn module it falls back to unaided practice with the model held until after the attempt. A class where D is at or below zero is left with the person. A class where A exceeds H_p and J_p sits at A is a candidate for replacement, meaning the model answers and the person reviews, and only outside the Learn module. The rule is applied per person, since the same class can fall in different cells for different people, and the cells are re-read after each retested probe.

8 Calibration

The central claims are the definitions, and their calibration is the instrument’s test suite and the Lean listing. The tests check that the freeze is deterministic across builds, that pairs match on tier and model correctness, the guess correction at chance and at a perfect run, the four statistics on

Table 2: What Research OS does with a task class, by the signs of the gain and of learning.

Gain D	Learning L	Action
> 0	≥ 0	augment: model answer in view after the person’s attempt
> 0	< 0	augment as a means only; unaided in Learn
≤ 0	any	leave with the person
$A > H_p, J_p \approx A$	any	candidate to replace outside Learn; person reviews

fixed data, the pair bootstrap on fixed data under a fixed seed, that no feedback is shown before the retest, and that the retest is gated by an injected clock [empirical: `packages/bkt/src/hai`, 2026-10-07, git show dev:packages/bkt/src/hai]. The Lean facts behind the definitions are listed in Appendix B. The error budget on a reported statistic is its bootstrap interval, and Section 6 gives the item counts that interval needs.

9 Limitations

No data. No probe has been retested, so nothing here has been measured on a person. The bank, the matching and the statistics have been exercised by tests on fixed data only.

One model, one run. A is one run of one model at one effort setting. A different model, a higher effort or a second sample would give a different A and a different pairing. The bank version and the model id are stored with every score so that a re-run is a new instrument version and never a silent change.

Four-choice items. The bank measures recognition under four fixed choices. It cannot show divergence, and it rewards a model that is good at eliminating distractors. The open-item bank of Section 7.4 is unbuilt.

Ceilings, drift, regression and practice. Four artefacts of a repeated test can mimic or hide an effect. A ceiling: when J_p and A both sit near the task maximum, D cannot show a gain and m_p collapses toward 1; the bank’s tiers run from recall to teach so that each probe carries items the model and the person miss, and a pair group in which the model is wrong is matched on purpose. Task drift: a probe draws items the person has never seen, so two probes a week apart test different items, and a change in H_p between probes mixes the person’s change with the items’ difficulty; the design answers this with matching on tier and model correctness inside each probe, so every within-probe statistic compares items of the same difficulty, and item-level difficulty matching across probes, from the pooled item accuracy, is planned before any trend is read. Regression to the mean: a person who scored low on day 0 by chance will tend to score higher on day 7, which inflates R_p ; the retest reuses the same 40 items, so R_p is a within-item difference and the chance component is bounded by the bootstrap interval, and the planned control arm, a probe with no AI on either half, gives the expected R_p under retest alone. Practice: the weekly unaided retest is itself a retrieval event that raises later recall [6]; the design reads L as the difference between the pair and solo halves at the same retest, so the practice effect falls on both halves, and parallel item forms, a second frozen bank with the same atoms and new distractors, are planned so that a later probe can test an atom without repeating its item.

The three connecting rules. The frontier draw, the predicted-recall report and the solo-only mastery update of Section 5 are all design; the probe draws by tier and model correctness today and holds no FSRs or mastery code [empirical: packages/bkt/src/hai, 2026-10-07, git show dev:packages/bkt/src/hai]. Until the Academy’s prerequisite graph is bound to the bank’s atoms by version, a probe cannot tell a frontier item from a known one. The learning system’s theorems hold under A1 through A5, and A1, that every encoded edge is mandatory and none is missing, is a property of the curriculum that no proof can supply.

Synthetic evaluation only. The 57,060 graph cases are every DAG on at most five nodes; the planner is untested above that size except by its fixtures. The statistical experiment detects what its own generator planted and says nothing about a real learner or a real item bank.

Seven days. The retest interval is fixed at seven days. Retention at thirty days or ninety is a different quantity, and a single interval cannot separate forgetting from a spacing effect.

Self-selection and the flagged items. Participants are users of a terminal app who opt in; the education strata will reflect that. The 68 flagged items are out until a person clears them, and the automated review is a set of surface rules, so an item with a subtler cue can pass it.

The decision rule. Table 2 is a rule with no result behind it yet. Its cells are set by the signs of two statistics and say nothing about effect size; a positive D of 0.02 and one of 0.30 fall in the same cell. Thresholds on size wait for the first intervals.

Licences. The item bank is derived from the Academy pack, whose lesson text is open and whose sources are listed per atom in the pack; the probe code is MIT with the repository. Per-person data never leaves the device, and the planned cross-person analysis uses only exported, consented records.

References

- [1] Fabrizio Dell’Acqua et al. “Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of Artificial Intelligence on Knowledge Worker Productivity and Quality”. In: *SSRN Electronic Journal* (2023). DOI: [10.2139/ssrn.4573321](https://doi.org/10.2139/ssrn.4573321).
- [2] Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. *Generative AI at Work*. Tech. rep. w31161. National Bureau of Economic Research, 2023. DOI: [10.3386/w31161](https://doi.org/10.3386/w31161).
- [3] Michelle Vaccaro, Abdullah Almaatouq, and Thomas Malone. “When combinations of humans and AI are useful: A systematic review and meta-analysis”. In: *Nature Human Behaviour* 8.12 (2024), pp. 2293–2303. DOI: [10.1038/s41562-024-02024-1](https://doi.org/10.1038/s41562-024-02024-1).
- [4] Hamsa Bastani et al. “Generative AI without guardrails can harm learning: Evidence from high school mathematics”. In: *Proceedings of the National Academy of Sciences* 122.26 (2025). DOI: [10.1073/pnas.2422633122](https://doi.org/10.1073/pnas.2422633122).
- [5] Gagan Bansal et al. “Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance”. In: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, 2021, pp. 1–16. DOI: [10.1145/3411764.3445717](https://doi.org/10.1145/3411764.3445717).

- [6] Henry L. Roediger and Jeffrey D. Karpicke. “Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention”. In: *Psychological Science* 17.3 (2006), pp. 249–255. DOI: [10.1111/j.1467-9280.2006.01693.x](https://doi.org/10.1111/j.1467-9280.2006.01693.x).
- [7] Anil R. Doshi and Oliver P. Hauser. “Generative AI enhances individual creativity but reduces the collective diversity of novel content”. In: *Science Advances* 10.28 (2024). DOI: [10.1126/sciadv.adn5290](https://doi.org/10.1126/sciadv.adn5290).
- [8] B. Efron. “Bootstrap Methods: Another Look at the Jackknife”. In: *The Annals of Statistics* 7.1 (1979), pp. 1–26. DOI: [10.1214/aos/1176344552](https://doi.org/10.1214/aos/1176344552).

A Glossary

unaided score https:

[//github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts](https://github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts)

H_p , the guess-corrected share of solo items a person gets right with no AI answer shown; see Definition 3.2.

AI score https:

[//github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/score.ts](https://github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/score.ts)

A , the guess-corrected share of the same frozen items the model gets right on its own; see Definition 3.2.

joint score https:

[//github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts](https://github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts)

J_p , the guess-corrected share of pair items a person gets right after the model’s answer and rationale are shown; see Definition 3.2.

multiplier https:

[//github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts](https://github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts)

$m_p = J_p / \max(H_p, A)$, the joint score over the better single agent; see Definition 3.5.

retention https:

[//github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts](https://github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts)

$R_p = H_p(t + 7d) - H_p(t)$, the change in the unaided score on the same items after seven days; see Definition 3.6.

dependence https:

[//github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts](https://github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/stats.ts)

The state in which a tool raises the joint score while the person keeps none of the gain a week later; see Definition 3.7.

probe https:

[//github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/probe.ts](https://github.com/gianyrox/bucket-foundation/blob/dev/packages/bkt/src/hai/probe.ts)

One session of 40 unseen items in 20 matched pairs, one item of each pair solo and one with AI, retested unaided seven days later; see Definition 3.3.

prerequisite closure <https://github.com/gianyrox/bucket-foundation/blob/dev/learning/research-os/learning-system/lean/LearningSystem.lean>

$C(t)$, the target t together with every concept on a prerequisite path into t ; see Definition 4.1.

required set <https://github.com/gianyrox/bucket-foundation/blob/dev/learning/research-os/learning-system/lean/LearningSystem.lean>

$R(t, M) = C(t) \setminus M$, the concepts in the closure the learner has not mastered; see Definition 4.2.

frontier <https://github.com/gianyrox/bucket-foundation/blob/dev/learning/research-os/learning-system/lean/LearningSystem.lean>

The ready concepts: unknown concepts whose immediate prerequisites are all known; see Definition 4.1.

coverage <https://github.com/gianyrox/bucket-foundation/blob/dev/learning/research-os/learning-system/brief/BRIEF.md>

G , the share of the catalog with a confirmed flag, with its axis coordinates g_j and radial extent r ; see Definition 4.10.

retrievability <https://github.com/gianyrox/bucket-foundation/blob/dev/src/lib/academy/fsrs.ts>

<https://github.com/gianyrox/bucket-foundation/blob/dev/src/lib/academy/fsrs.ts>

$R(t, S) = (1 + Ft/S)^{-1/2}$ with $F = 0.9^{-2} - 1$, the FSRS-5 predicted recall t days after a review at stability S ; see Definition 4.13.

mastery <https://github.com/gianyrox/bucket-foundation/blob/dev/src/lib/academy/mastery.ts>

$M = P^\alpha R^\beta$ with proficiency P , FSRS retention R at 90 days and $\alpha = \beta = 1$; see Definition 4.14.

B Lean listings

`lean/Bucket/Multiplier.lean`, checked by `make lean`, holds the four facts the method leans on. The guess-corrected score is carried as an integer numerator $4c - n$ over the denominator $3n$.

```
def RETEST_MS : Nat := 7 * 24 * 60 * 60 * 1000
def retestDue (t : Nat) : Nat := t + RETEST_MS
theorem retestDue_gt (t : Nat) : t < retestDue t

structure Probe where
  correct : Nat
  items : Nat
  bounded : correct <= items

def Probe.numerator (p : Probe) : Int := 4 * (p.correct : Int) - p.items
def Probe.denominator (p : Probe) : Int := 3 * (p.items : Int)
theorem Probe.numerator_le_denominator (p : Probe) : p.numerator <= p.denominator
theorem Probe.numerator_ge_neg_items (p : Probe) : -(p.items : Int) <= p.numerator
theorem Probe.numerator_zero_at_chance (p : Probe) (h : 4 * p.correct = p.items) :
  p.numerator = 0

structure Interval where
  lo : Int
  hi : Int
```

```

def dependence (gain learning : Interval) : Prop := 0 < gain.lo /\
  learning.lo <= 0
theorem dependence_needs_gain (gain learning : Interval) (h : dependence
  gain learning) :
  0 < gain.lo
theorem no_dependence_when_learning_kept (gain learning : Interval) (h :
  0 < learning.lo) :
  ~ dependence gain learning

```

lean/Bucket/Learning.lean is a verbatim copy of learning/research-os/learning-system/lean/LearningSystem.lean. Beyond the four statements quoted in Section 4.2, it holds the following, each proved; After, Path, Coverage and the four-element Diamond type are defined in the source as Section 4.1 states them.

```

def Enumerates (edge : a -> a -> Prop) (known : a -> Prop) (target : a)
  (remaining : List a) : Prop :=
  remaining.Nodup /\ forall v, v in remaining <=> Required edge target v
  /\ ~ known v

def Earlier (edge : a -> a -> Prop) (known : a -> Prop) (order : List a)
  : Prop :=
  forall before v rest, order = before ++ v :: rest -> forall p, edge p
  v -> known p \/ p in before

theorem restricted_closed : Closed edge (RestrictedKnown edge target
  known)
theorem required_known (hc : Closed edge known) (ht : known target)
  (hr : Required edge target v) : known v
theorem after_closed (hc : Closed edge known) (hp : Path edge known
  steps) :
  Closed edge (After known steps)
theorem distance_antitone (growth : forall x, small x -> large x)
  (oldEn : Enumerates edge small target oldOrder)
  (newEn : Enumerates edge large target newOrder) : newOrder.length <=
  oldOrder.length

theorem coverage_monotone (growth : forall v, small v = true -> large v
  = true) :
  Coverage catalog small weight <= Coverage catalog large weight
theorem squared_extent_monotone (growth : CoordinateGrowth small large)
  :
  SquaredExtent small <= SquaredExtent large

theorem diamond_minimum : forall steps, Path diamondEdge (fun _ => False
  ) steps ->
  After (fun _ => False) steps .target -> diamondOrder.length <= steps
  .length
theorem diamond_chain_insufficient : ~ Path diamondEdge (fun _ => False)
  diamondChain

```

lake build output, quoted verbatim [empirical: papers/human-ai-multiplier/lean, 2026-10-07, make lean].

Build completed successfully (5 jobs).